# SentryScan: Agent Architecture & Environment Specification

**Course:** 21CSE525T — Agent Technology
**Assignment:** 1.1 — PEAS & Environment Modelling
**Student:** R Ganesh [RA2212704010031]
**Repository:** https://github.com/drgeekg/SentryScan
**Status:** Submitted
**Git Tag:** `assignment-1.1`

## 1. Persona & Problem Statement

### 1.1 Problem Statement

Modern enterprise infrastructure produces massive volumes of security telemetry—service banners, port states, TLS handshakes, and access logs across thousands of endpoints. Security operations centers (SOCs) face two chronic bottlenecks: 1. **Alert Fatigue & Noisy Prioritization:** Uncorrelated alerts overwhelm human operators; high-severity critical exposures get buried under thousands of low-impact notices. 2. **Operational Disruption of Live Probing:** Active vulnerability probing (e.g., intrusive scanning, fuzzing, packet flooding) risks taking down delicate operational technology (OT) or violating legal and organizational compliance boundaries.

There is an acute need for an **autonomous, non-intrusive security analysis agent** capable of ingesting streaming endpoint telemetry, validating data integrity, detecting high-risk exposures through deterministic reasoning, and deliberating over threat posture without active live scanning.

### 1.2 Agent Persona: SentryScan

**SentryScan** is an authorized, defensive security analyst agent designed to simulate intelligent vulnerability triage.

- **Role:** Autonomous Defensive Telemetry Analyst.
- **Core Philosophy:** SentryScan operates strictly on *perceived telemetry* rather than active exploitation. It acts as an observant, rule-grounded, and deliberative defender.
- **Operational Scope:**
- Ingests structured synthetic telemetry records representing network services and host metadata.
- Distinguishes authentic telemetry from poisoned or corrupted records without access to ground-truth answer keys.
- Applies explicit condition–action rules to identify confirmed exposures (e.g., deprecated protocols, default credentials, exposed administrative interfaces).
- Prioritizes remediation based on explicit multi-attribute utility and updates internal mental states (BDI) as threat context evolves.

### 1.3 Safety Boundary & Ethical Simulation Mandate

To maintain strict ethical boundaries and academic reproducibility: - **Synthetic Simulation Only:** SentryScan does **not** perform network-level packet craft, port scanning, credential stuffing, or exploit execution. - **Documentation Address Space:** All host addresses strictly use IETF documentation ranges (RFC 5737: `192.0.2.0/24`, `198.51.100.0/24`, `203.0.113.0/24`; RFC 3849: `2001:db8::/32`). SentryScan never attempts socket connections to these addresses. - **Untrusted Telemetry Guarantee:** All banners and external strings are treated as untrusted data inputs, precluding prompt-injection side channels or unvalidated execution.

---

## 2. PEAS Framework Formulation

The formal PEAS (Performance, Environment, Actuators, Sensors) specification for SentryScan defines its operational boundaries and rational agency:

| Component | Specification for SentryScan |
|---|---|
| **Performance Measure ($P$)** | - **Detection Accuracy:** High precision and recall on genuine exposures without false alarms on benign services.<br>- **Data Integrity Defense:** Detection rate of poisoned/mutated records (stale timestamps, banner contradictions, rate bursts) without access to ground truth.<br>- **System Stability & Latency:** Minimal queue turnaround time; bounded memory footprint; zero dropped events.<br>- **Protocol Compliance:** Handshake resolution rate in multi-agent negotiation; bounded rounds ($\le 5$); zero unauthorized message acceptances.<br>- **Audit Completeness:** 100% gap-free event sequence persistence ( `seq` ). |
| **Environment ($E$)** | - **Simulated Endpoint Fleet:** Heterogeneous synthetic hosts ( `sim-host-001` … `sim-host-N` ) generating concurrent telemetry.<br>- **Telemetry Stream:** Ingestion queue carrying normal baseline records, malformed payloads, and deliberately mutated (poisoned) records.<br>- **Communication Channels:** Loopback TCP sockets ( `127.0.0.1` ) and in-process message queues.<br>- **Knowledge Repositories:** Read-only CVE / NVD data interfaces and local offline caches. |
| **Actuators ($A$)** | - **Finding Publisher:** Emits structured `Finding` objects with deterministic severity, evidence links, and confidence scores.<br>- **Event Recorder:** Commits append-only chronological `AgentEvent` entries with monotonic `seq` to JSONL and SQLite sinks.<br>- **Envelope Dispatcher:** Constructs, signs (Ed25519), and transmits inter-agent messages ( `ALERT` , `PARAMS_UPDATE` , `CFP` , `PROPOSE` , `ACCEPT` ).<br>- **BDI Intention Actuator:** Modifies active goals, triggering plan revision or resource reallocation.<br>- **Report Actuator:** Generates structured JSON/CSV audit reports and Rich console dashboards. |
| **Sensors ($S$)** | - **Telemetry Sensor:** Stream receiver that parses and validates raw telemetry into typed `TelemetryRecord` objects.<br>- **Integrity Sensor:** Rule-based parser checking timestamp drift, hash duplication, service-banner mismatch, and prompt injection strings.<br>- **Socket / Network Sensor:** TCP frame listener decoding length-prefixed JSON envelopes and validating security credentials.<br>- **Knowledge Sensor:** Client querying local CVE cache and external NVD endpoints for vulnerability enrichment. |

## 3. Environment Classification (Russell & Norvig Taxonomy)

SentryScan's operational environment is categorized along the seven classical dimensions established by Russell & Norvig (*Artificial Intelligence: A Modern Approach*):

| Dimension | Classification | Theoretical Justification & SentryScan Grounding |
|---|---|---|
| **Observability** | **Partially Observable** | The agent observes only surface-level telemetry emitted by devices (ports, service names, version banners). It cannot inspect remote filesystem states, kernel memory, or underlying device internals. Furthermore, ground-truth poisoning labels are held in an evaluator-only store, preventing the agent from directly observing whether a record has been tampered with. |
| **Determinism** | **Stochastic** | Although SentryScan's internal reasoning engine is strictly deterministic (same seed produces identical rule firings), the external telemetry environment contains stochastic elements: variable queue latency, concurrent thread interleaving, synthetic network jitter, and probabilistic injection operators applied by the fleet generator. |
| **Episodic vs. Sequential** | **Sequential** | SentryScan is fundamentally sequential. Current perceptual decisions directly constrain future states: identifying a suspect record updates the device baseline; creating a finding alters the BDI belief store; and awarding an analysis batch changes the budget available for subsequent negotiation rounds. |
| **Static vs. Dynamic** | **Dynamic** | The environment evolves continuously while the agent deliberates. The background fleet daemon continues to generate telemetry, populate queues, and update device profiles while scanner workers process historical batches. The agent cannot assume the environment remains frozen during decision computation. |
| **Discrete vs. Continuous** | **Discrete** | Percepts, state variables, and actions are discrete. Network ports are integers ($1 \dots 65535$), performatives belong to a finite enumeration ( `ALERT` , `CFP` , `PROPOSE` , etc.), vulnerability severities are categorical ( `LOW` , `MEDIUM` , `HIGH` , `CRITICAL` ), and event sequences follow discrete monotonic steps ($seq \in \mathbb{N}$). |
| **Single-Agent vs. Multi-Agent** | **Multi-Agent** | SentryScan operates as a multi-agent system from Unit 3 onwards. The supervisor coordinates multiple specialized agents (Scout, Triage, Scanner Workers, and Analyst) with partially conflicting utility functions (Scout seeks breadth of host coverage; Triage seeks depth of critical risk). They engage in Contract-Net negotiation to achieve task allocation. |
| **Known vs. Unknown** | **Known** | The structural laws of the environment—telemetry schemas, rule definitions, network protocols, and utility equations—are completely specified and known to the agent design. The state of individual devices, however, must be discovered through telemetry sensing. |

## 4. Agent Architecture vs. Conventional Object-Oriented Programming (OOP)

A fundamental theoretical pillar of Unit 1 (Agent Technology) is distinguishing autonomous agent architectures from conventional Object-Oriented Programming paradigms (Russell & Norvig, Chapter 2; Wooldridge, *An Introduction to MultiAgent Systems*).

| Conventional OOP Paradigm | Autonomous Agent Paradigm (SentryScan) |
|---|---|
| Invocation: Passive method dispatch (caller forces execution) | Interaction: Autonomous reaction & deliberate acceptance of messages |
| State: Internal variables & encapsulated data structures | State: Cognitive mental models (Beliefs, Desires, Intentions) |
| Environment: Abstracted away; closed-world heap execution | Environment: Continuously sensed, situated in stochastic world |
| Communication: Rigid typed RPC method signatures | Communication: Semantic speech-act performatives (CFP, PROPOSE, ALERT) |
| Goal: Deterministic execution of predetermined algorithmic path | Goal: Flexible, rational pursuit of utility under resource constraints |

### 4.1 Comparative Analysis Matrix

| Architectural Dimension | Conventional OOP | Autonomous Agent Paradigm (SentryScan) |
|---|---|---|
| **Control Autonomy** | **Subordinate:** An object has no choice over whether to execute an invoked method. If `scanner.scan_host()` is called, execution proceeds unconditionally. | **Autonomous:** SentryScan agents own their thread of control. When sent a `CFP` (Call for Proposal), the agent evaluates its resource budget, current workload, and utility equation before choosing whether to bid, pass, or counter-propose. |
| **Environmental Grounding** | **Unsituated:** Objects reside in transient memory heaps. They do not maintain an ongoing, embodied relationship with an evolving external world. | **Situated:** SentryScan is continually grounded in a simulated fleet environment via sensors (telemetry streams, socket listeners) and actuators (event logs, finding publishers). |
| **State Representation & Deliberation** | **Static Data Structures:** Object state consists of primitives and collections (e.g., `boolean is_vulnerable`). | **Cognitive Modalities (BDI):** SentryScan maintains epistemic *Beliefs* (fleet posture, network topology), motivational *Desires* (maximize coverage, surface critical vulnerabilities), and committed *Intentions* (active inspection plans). |
| **Interaction Semantics** | **Method Invocations:** Tight synchronous coupling via method calls with fixed parameter lists. | **Speech-Act Performatives:** Declarative, loosely coupled message envelopes governed by FIPA-compliant performatives ( `ALERT` , `CFP` , `PROPOSE` , `ACCEPT` , `REJECT` ). |
| **Resilience & Reconsideration** | **Exception Handling:** Rigid control flow via `try-catch` blocks that unwind stacks on errors. | **Dynamic Re-planning:** SentryScan detects material belief changes (e.g., an unexpected critical vulnerability or an integrity breach) and autonomously triggers intention reconsideration. |

### 4.2 Explicit Unit-1 Paradigm Synthesis

In the terminology of the Unit-1 agent taxonomy, SentryScan embodies a progressive synthesis: 1. **Simple Reflex Agent (Assignments 1.2, 2.1):** Operates on immediate condition-action rules: $$\text{Rule}: \text{Percept}(service = \text{Telnet}) \implies \text{Action}(\text{Flag}(\text{Unencrypted_Protocol}))$$ 2. **Goal- and Utility-Based Agent (Assignments 3.1, 3.2):** Balances multiple competing outcomes using an explicit mathematical utility formulation: $$U(o) = w_1\,C_{severity}(o) + w_2\,C_{hosts}(o) - w_3\,\text{Cost}(o) - w_4\,\text{Time}(o) - w_5\,\text{Risk}(o)$$ 3. **Deliberative BDI Agent (Assignments 4.1, 5.2):** Implements Bratman's Belief-Desire-Intention cognitive framework, achieving rational balance between commitment to active plans and reactive replanning upon environmental disruption.

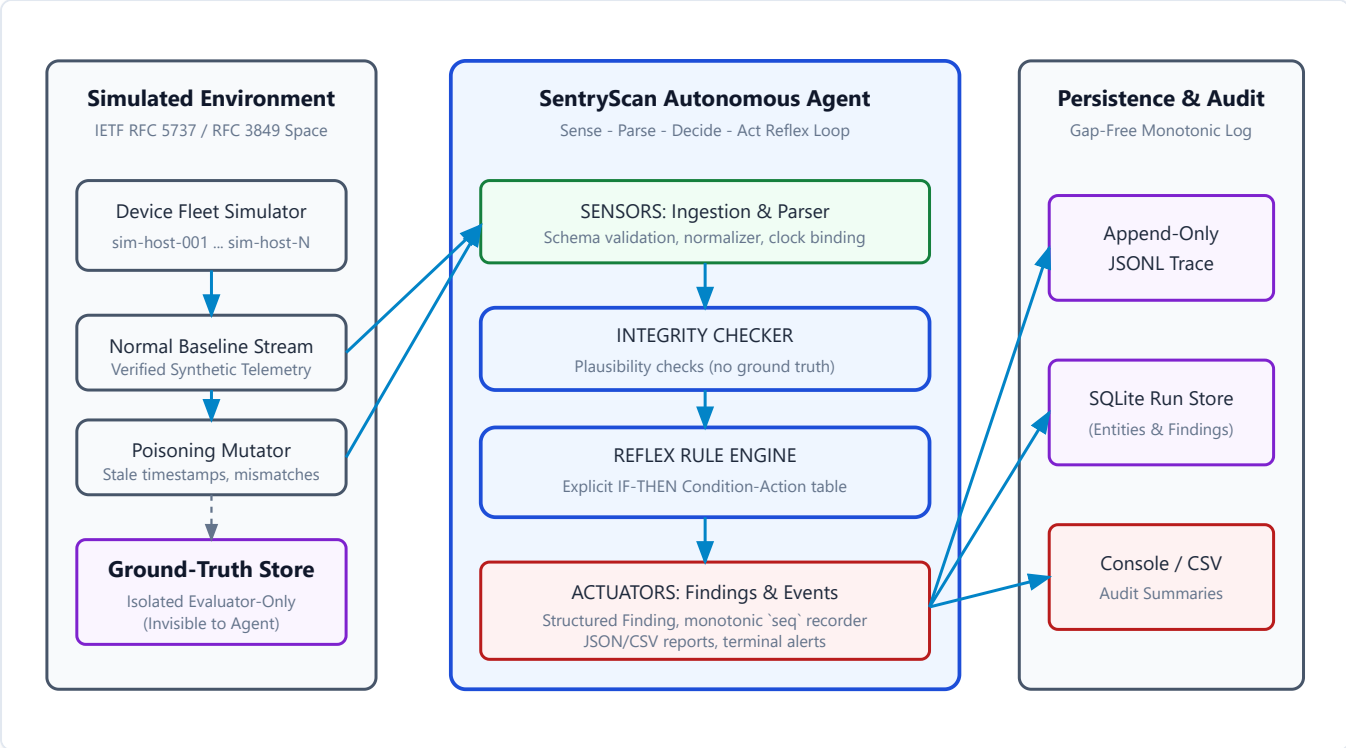
---

## 5. Initial Architecture & Data Flow

SentryScan's initial architecture is grounded in a single-agent Sense–Parse–Decide–Act reflex loop, backed by an isolated environment simulator and a gap-free event persistence pipeline.

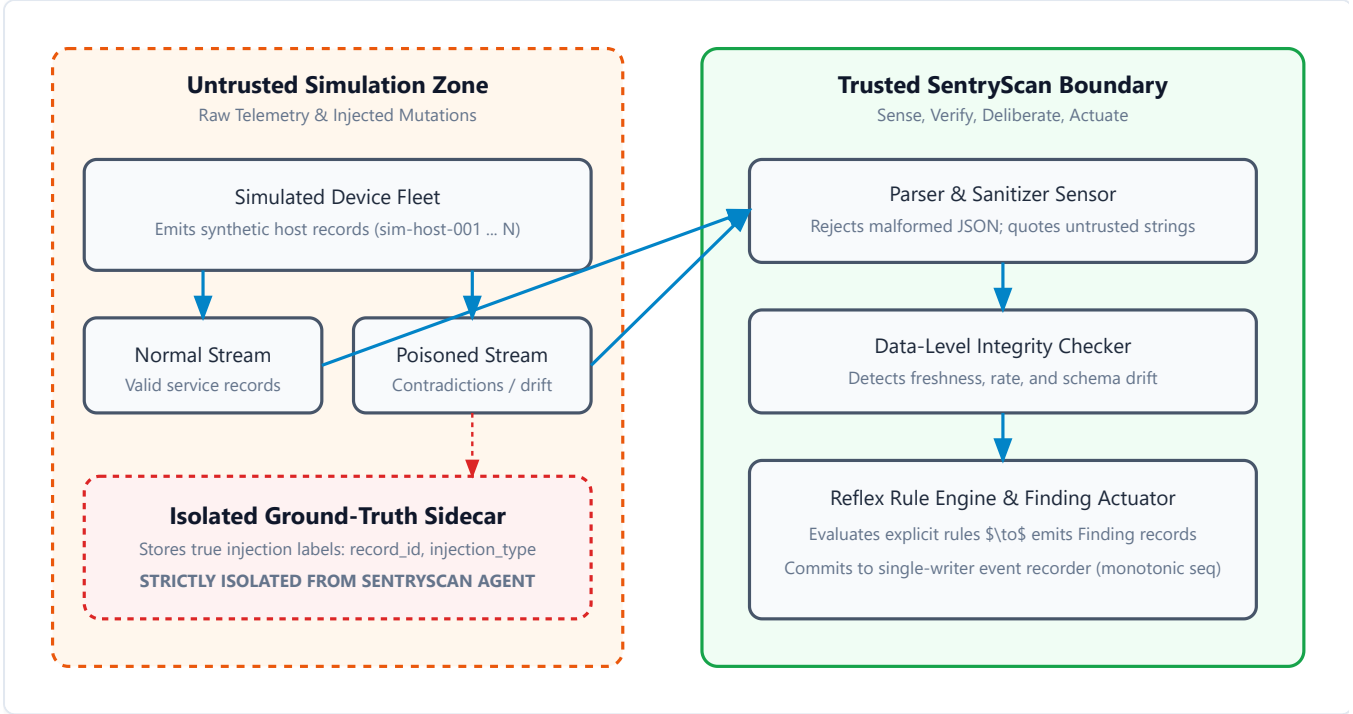
### 5.1 Architecture Diagram

The structural flow between the simulated fleet, SentryScan's internal reasoning components, and the audit persistence sinks is shown below:




### 5.2 Environment and Trust Boundary Model

To ensure rigorous security evaluation without contamination, the system enforces a strict isolation boundary between the generator's ground-truth mutations and the agent's percept pipeline:




### 5.3 Step-by-Step Processing Pipeline

1. **Telemetry Sensing (`Sensor`):**
2. The device fleet emits JSON records over an in-memory queue.
3. The ingestion sensor decodes the payload, validates types, and bounds port values ($1 \dots 65535$). Malformed payloads trigger an immediate `record_rejected` audit event.
4. **Plausibility & Integrity Inspection (`IntegrityChecker`):**
5. SentryScan evaluates data-level sanity: timestamp freshness relative to the injected `Clock`, port-to-service plausibility, and duplicate payload hashes.
6. Suspect records are tagged with an `integrity_flags` list rather than silently discarded, preserving forensic traceability.
7. **Reflex Rule Evaluation (`RuleEngine`):**
8. The parsed percept is matched against an explicit condition–action table.
9. Every match emits a deterministic `Finding` record with immutable severity (`CRITICAL`, `HIGH`, `MEDIUM`, `LOW`) and confidence scores.
10. **Audit Logging & Actuation (`Actuators`):**
11. The single-writer `EventRecorder` assigns a monotonic, gap-free sequence number (`seq`).
12. The event is appended to `data/runs/{run_id}/trace.jsonl` and prepared for SQLite indexing, ensuring full retrospective reproducibility.

---

## 6. Future Multi-Agent & Deliberative Extensions

While Assignment 1.1 focuses on foundational modeling and the single-agent reflex loop, the architecture incorporates explicit expansion seams designed for subsequent course units:

```
Assignment 1.1 (Foundations)
       │
       ▼
Assignment 1.2: Deterministic Rule Engine & JSONL Recorder
       │
       ▼
Assignment 2.1: Concurrent Fleet Daemon & Integrity Checker
       │
       ▼
Assignment 2.2: Two-Process Architecture over TCP Socket Transport
       │
       ▼
Assignment 3.1: LangGraph Supervisor & Multi-Agent Negotiation (Scout vs. Triage)
       │
       ▼
Assignment 3.2: Explicit Mathematical Utility Optimization
       │
       ▼
Assignment 4.1: Deliberative BDI Cognitive Architecture (Analyst Agent)
       │
       ▼
Assignment 4.2: Vulnerability Knowledge Ontology & NVD Enrichment
       │
       ▼
Assignment 5.1: Ed25519 Cryptographic Envelope Authentication
       │
       ▼
Assignment 5.2: Capstone Integration & Real-Time Browser Dashboard
```

1. **Concurrency and Asynchronous Queues (Unit 2):** Ingestion shifts to a non-blocking background daemon using `threading` and `queue.Queue`, guarded by mutex locks against race conditions.
2. **Distributed Process Topology (Unit 2):** The Analyst agent decouples into a separate process communicating over length-prefixed TCP localhost sockets.
3. **Multi-Agent Coordination via LangGraph (Unit 3):** The monolithic supervisor coordinates autonomous subagents ( `ScoutAgent` prioritizing coverage; `TriageAgent` prioritizing critical risk) utilizing Contract-Net negotiation ( `CFP` $\to$ `PROPOSE` $\to$ `COUNTERPROPOSE` $\to$ `ACCEPT` / `REJECT` ).
4. **Utility-Driven Decision Optimization (Unit 3):** Bids are evaluated through an explicit, normalized multi-attribute objective function under hard budget constraints.
5. **Belief-Desire-Intention (BDI) Deliberation (Unit 4):** The Analyst agent maintains epistemic beliefs and reconsider intentions when high-severity alerts arrive.
6. **Cryptographic Identity & Security Gate (Unit 5):** Inter-agent messages are signed using Ed25519 over canonical byte representations with anti-replay nonce tracking.